Notification Platform — Build Plan

Oct 9, 2026 · @Gerick

Two decisions: the .NET version and the API documentation

Use .NET 10 with C# 14.

- .NET 10 is a Long Term Support release, supported until 14 November 2028.
- .NET 8 and .NET 9 both lose support on 10 November 2026, a month from now, so starting on either one means upgrading before you finish.
- .NET 11 arrives in November 2026, but odd-numbered releases are Standard Term Support with two years of support. Choose the LTS release for something you want to keep running.
- Pin it so every machine and the CI build the same way: a `global.json` file selects SDK 10.0.x, and `Directory.Build.props` sets `net10.0` for every project.

Use OpenAPI as the contract, shown through Scalar.

OpenAPI, Swagger UI and Scalar are not three competing choices. OpenAPI is the specification: a JSON document that describes your API. Microsoft's built-in package and Swashbuckle generate that document, and Swagger UI and Scalar display it.

Job	Choice	Why
Generate the document	`Microsoft.AspNetCore.OpenApi`, Microsoft's own package: `AddOpenApi()` and `MapOpenApi()` serve the document at `/openapi/v1.json`, in Development only by default	Maintained by the ASP.NET team and works with controllers and minimal APIs, so your contract has no third-party dependency
Browse and try requests	Scalar (`Scalar.AspNetCore`, MIT): `MapScalarApiReference()` serves it at `/scalar`	A request builder, code samples in several languages and search, all reading the same document

Job	Choice	Why
Export for tools	`Microsoft.Extensions.ApiDescription.Server` with `OpenApiGenerateDocumentsOnBuild`	Writes the document at build time, so CI can diff it for breaking changes and generate the React client from it

- Swagger UI is also MIT-licensed and perfectly good. Both viewers read the same document, so switching later costs one line. Scalar is the pick because exploring and sending requests is nicer.
- Skip Swashbuckle for a new project. The ASP.NET Core Web API template now generates the document with the built-in package.
- Keep controllers, as in your tree. The OpenAPI package describes controller-based and minimal APIs alike.
- Leave the document and Scalar off in production. Expose them in Development and staging only.

Backend solution structure

This is your tree adjusted to the v2.0 design. You do not create everything on day one: each project appears in the week that needs it, listed in the table below the tree.

```
NotificationPlatform/
├── NotificationPlatform.sln
├── global.json                 # pins the .NET 10 SDK
├── Directory.Build.props       # net10.0, nullable on, warnings as errors
├── Directory.Packages.props    # one place for every NuGet version
├── .editorconfig
├── docker-compose.yml
│
├── docs/
│   ├── adr/                    # one short decision record per decision
│   ├── notes/                  # your weekly learning log
│   └── runbooks/
│
├── infra/
│   ├── keycloak/               # realm export
│   ├── kafka/                  # topic creation script
```

```
|   └── observability/           # Prometheus config, alert rules
|
├── src/
|   ├── Lib/                     # contracts only; references nothing
|   |   ├── Contracts/          # Kafka message envelope, API
request/response
|   |   ├── DTOs/
|   |   ├── Enums/              # NotificationStatus, FailureClass
|   |   └── Common/             # IClock, correlation ID, Result<T>
|   |
|   ├── Domain/                 # pure rules: no I/O, no frameworks
|   |   ├── Notifications/      # state machine and transition rules
|   |   └── Retry/              # failure classes, backoff with jitter
|   |
|   ├── Data/                   # EF Core + PostgreSQL, shared by every
service
|   |   ├── Entities/
|   |   ├── Configurations/
|   |   ├── Migrations/
|   |   └── Repositories/
|   |
|   ├── Templating/             # Fluid sandbox, JSON Schema variable checks
|   ├── Migrator/               # console app: apply migrations, then exit
|   |
|   ├── NotificationSenderApi/  # public API: accept and store, never
touches Kafka
|   |   ├── Controllers/
|   |   ├── Services/
|   |   ├── Authentication/
|   |   ├── Middleware/         # correlation ID, problem details,
idempotency
|   |   └── Configuration/
|   |
|   ├── NotificationWebApi/     # admin API behind the dashboard
|   |   ├── Controllers/
|   |   ├── Services/
|   |   ├── Authentication/
|   |   ├── Middleware/
|   |   └── Hubs/               # SignalR
|   |
|   ├── NotificationWorker/
|   |   ├── Outbox/             # relay: outbox rows to Kafka
|   |   ├── Consumers/
```

```
|   |   ├── EmailProcessing/     # render and send with MailKit
|   |   ├── Retry/               # retry-topic consumers
|   |   └── DeliveryTracking/
|   |
|   └── LoadTestWorker/
|       ├── TestOrchestration/
|       ├── K6/
|       ├── Metrics/
|       └── Results/
|
├── tests/
|   ├── NotificationPlatform.UnitTests/
|   └── NotificationPlatform.IntegrationTests/   # Testcontainers: Postgres,
Kafka
|
├── load-tests/                  # k6 scripts are JavaScript, not C#
|   ├── scenarios/
|   ├── scripts/
|   └── configurations/
|
└── frontend/
    └── notification-dashboard/  # starts after the backend is working
```

What changed from your tree, and why

- `Kafka/` left NotificationSenderApi. The Sender API only writes to PostgreSQL, and the outbox relay in the worker publishes to Kafka, so a Kafka outage cannot stop requests being accepted.
- `Email/` moved into the worker, because sending is the worker's job. `Templates/` became a shared `Templating` project: the Sender API validates variables, the Web API previews, and the worker renders.
- `Repositories/` moved into `Data`, because three services read the same tables.
- `Domain` is new. It holds rules with no I/O, such as the state machine and retry classification. This is the easiest code to test and the code interviewers ask about.
- `Migrator` is new, so database migrations run as a job and not when an API starts.
- Tests moved out of `src/` into `tests/`, split into unit and integration projects. `LoadTestScenarios` became `load-tests/scenarios`, because k6 scenarios are JavaScript.
- `docs/` and `infra/` are new, so decisions, runbooks and configuration have a home.

Dependency rules

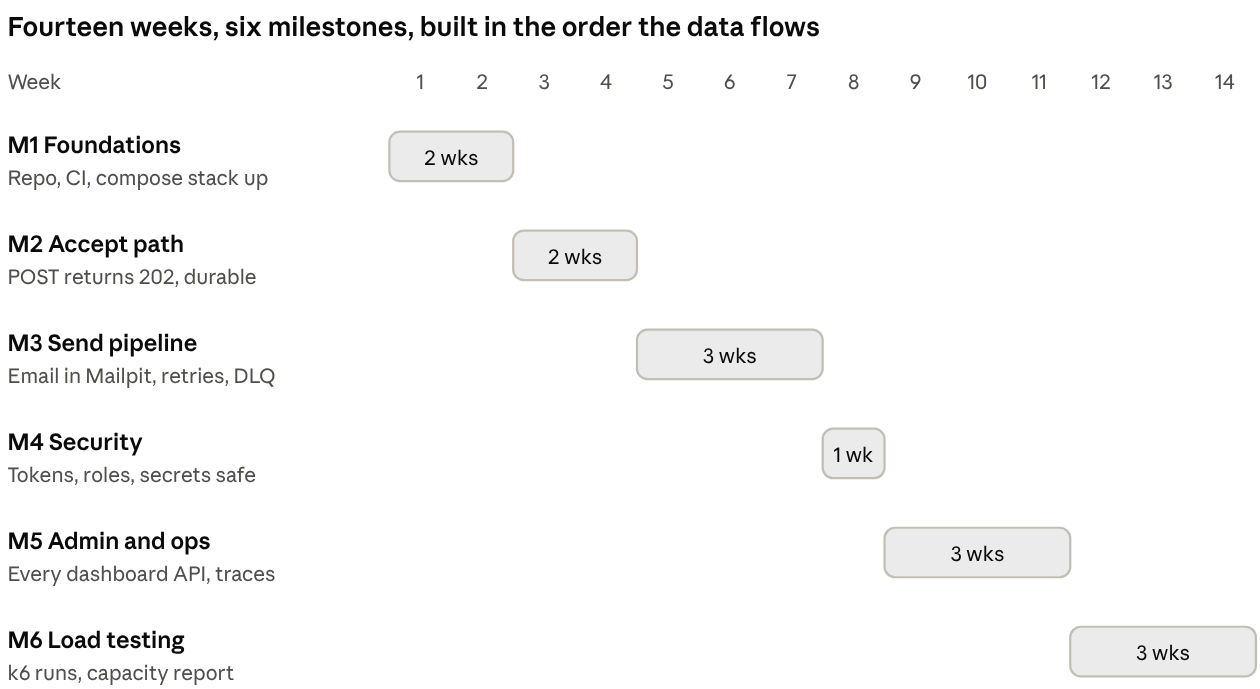
- `Lib` references nothing. `Domain` references `Lib`. `Data` references `Lib` and `Domain`. `Templating` references `Lib`.
- The services and the Migrator may reference those four projects, but never each other.
- Week 3 adds architecture tests that fail the build if a rule is broken.

When each project is created

Project	Created in	Why then
Lib, Domain, UnitTests	Weeks 1 and 2	The state machine is the first thing worth testing
Data, Migrator, Templating	Week 2	Every service needs the tables first
NotificationSenderApi, IntegrationTests	Week 3	The accept path is the smallest slice that proves the design
NotificationWorker	Week 5	It needs messages to relay, so the accept path comes first
NotificationWebApi	Week 9	Security (Week 8) comes before the admin API
LoadTestWorker	Week 12	It needs a system worth measuring
frontend	After Week 11	It is generated from the admin API's OpenAPI document

How the plan works

Build in the order the data flows, and add safety before breadth. Each milestone ends with something you can demonstrate.



build order · 6 milestones over 14 weeks

Why this order

1. Foundations and the data model come first, because every service reads the notifications table.
2. The accept path comes next. It is the smallest slice that proves the central idea: a request is accepted only once it is safely stored.
3. The send pipeline follows, so you see a message travel end to end (API, database, Kafka, worker, email) as early as possible.
4. Security comes before the admin API, so permissions are built in and not bolted on later.
5. The admin API and observability come next, because the dashboard needs both.
6. Load testing comes last, because it needs a system worth measuring. You still take a first baseline in Week 12, before any tuning, so the improvements are provable.
7. The frontend starts once the admin API exists, and is generated from its OpenAPI document.

How to work each day

- One day is one focused session of 2 to 3 hours. Day 5 of every week is a consolidate day for catching up, tests, a decision record and notes. If a day takes two sessions, take two; never skip its "done when" check.
- End every day with a commit and a push. Use Conventional Commit messages such as `feat: add idempotency keys`.
- Read for at most 20 minutes before building. Understanding comes from making the thing work and then explaining it.
- Keep a learning log in `docs/notes/week-NN.md`: three sentences each week on what you built, what surprised you, and what you would explain in an interview.
- Write a unit test the same day for any rule in `Domain`.
- Check the license of every NuGet package before adding it. The PRD's license audit lists the ones to avoid.
- If you are stuck for more than 30 minutes, write the question in your log, read the error message again, then bring me the code and the error.

Weeks 1 to 4: foundations, data and the accept path

Week 1: Foundations

Goal: a repository, a build pipeline and all the infrastructure running on your machine. **Learn:** what an LTS release is; what a Kafka topic, partition, offset and consumer group are; why containers.

- [ ] **Day 1 · Install and pin the tools.** Install the .NET 10 SDK, Docker, Git and an editor (Visual Studio, Rider, or VS Code with C# Dev Kit). Create the Git repository and add `global.json` pinning SDK 10.0.x. *Done when:* `dotnet --version` prints 10.0.x inside the repo and `docker run hello-world` works.
- [ ] **Day 2 · Solution and CI.** Run `dotnet new sln`, create the `Lib` and `Domain` class libraries under `src/`, and add `Directory.Build.props` (net10.0, nullable on, warnings as errors) and `Directory.Packages.props`. Add a GitHub Actions workflow that runs `dotnet build` and `dotnet test`. *Done when:* the workflow is green on a pushed commit.
- [ ] **Day 3 · PostgreSQL and Mailpit in Docker Compose.** Write `docker-compose.yml` with a PostgreSQL service (named volume) and Mailpit. *Done when:* you can connect to PostgreSQL with a SQL client, and Mailpit's web page on port 8025 shows a test email you sent to port 1025 with any SMTP test tool.

- [ ] **Day 4 · Kafka in KRaft mode.** Add a single-node Kafka (official Apache image, KRaft, no ZooKeeper). Use the console tools inside the container to create a topic with 3 partitions, produce 10 messages, consume them in a group, stop the consumer, produce more and restart it. *Done when:* the restarted consumer continues where it stopped and you can explain why (committed offsets).
- [ ] **Day 5 · Consolidate.** Write ADR-001 (.NET 10 LTS) and ADR-002 (PostgreSQL plus Kafka), a README with the one-command start, and your first learning-log note explaining partitions and consumer groups in your own words. *Done when:* `docker compose up` from a fresh clone starts everything.

Week 2: Domain and data

Goal: the core rules and the database exist, tested, before any API. **Learn:** state machines; compare-and-set; EF Core migrations; what Liquid templates are.

- [ ] **Day 1 · Notification states and the state machine.** Add `NotificationStatus` to `Lib/Enums` (Queued, Published, Sending, Retrying, SmtpAccepted, Delivered, Bounced, Failed, DeadLettered, Suppressed, Expired). In `Domain`, write `NotificationStateMachine` with a table of allowed transitions. Create the UnitTests project (xUnit). *Done when:* tests prove every legal transition passes and every illegal one is rejected.
- [ ] **Day 2 · The Data project and first entities.** Create `Data` with EF Core and Npgsql. Add entities Client, Template, TemplateVersion, Notification and OutboxMessage, with time-ordered UUIDv7 keys and a unique index on client plus idempotency key. *Done when:* `dotnet ef migrations add Initial` produces a migration whose SQL you have read and can explain table by table.
- [ ] **Day 3 · The Migrator.** Create the `Migrator` console app that applies migrations and exits, and run it in Compose as a one-shot service after PostgreSQL is healthy. *Done when:* `docker compose up` creates the tables in a fresh database, and running it twice changes nothing.
- [ ] **Day 4 · The Templating library.** Create `Templating` with Fluid: render subject, HTML and text from a template and a data dictionary, with auto-escaping on, a render timeout, an output size cap, and no access to .NET types. *Done when:* unit tests cover a normal render, a missing variable, a script-injection attempt, and a template that loops forever (it times out).
- [ ] **Day 5 · Consolidate.** Draw the entity diagram in Mermaid in `docs/`, add a seed script for one client and one template, and add a coverage report to CI. *Done when:* the diagram renders on GitHub and the seed script runs against a fresh database.

Week 3: The accept path

Goal: `POST` returns 202 only after the request is durable. **Learn:** HTTP semantics (202, 409, 422, 429); RFC 9457 problem details; database transactions; why an outbox exists.

- [ ] **Day 1 · Sender API skeleton.** Create `NotificationSenderApi` (ASP.NET Core Web API with controllers). Add Serilog JSON logging, `/health/live` and `/health/ready`, `AddOpenApi()` and `MapOpenApi()`, and Scalar. *Done when:* `/scalar` shows the API and `/health/ready` fails when PostgreSQL is stopped.
- [ ] **Day 2 · Request model, validation, errors.** Add `POST /api/v1.1/Email/SendEmail` with request and response models in `Lib/Contracts`. Validate recipients (1 to 50, syntax), size limits and required headers. Add middleware for the correlation ID and for RFC 9457 problem details. *Done when:* bad requests return 400 or 422 problem documents with a field-error list and a correlation ID, and every log line carries that ID.
- [ ] **Day 3 · One transaction, two rows.** On a valid request, insert a Queued notification and an OutboxMessage in a single EF Core transaction and return 202 with a `Location` header. Use a hard-coded client for now. *Done when:* one POST creates both rows, and forcing an exception between the two inserts leaves neither.
- [ ] **Day 4 · Idempotency.** Require the `Idempotency-Key` header and store it with a payload hash. The same key and payload returns the original response with `Idempotent-Replayed: true`; the same key with a different payload returns 409. Let the unique index settle two simultaneous requests. *Done when:* a test sending the same request 20 times in parallel creates exactly one notification.
- [ ] **Day 5 · Consolidate: first integration tests.** Create the IntegrationTests project with Testcontainers (PostgreSQL) and `WebApplicationFactory`. Turn the Day 3 and Day 4 checks into automated tests, add architecture tests (NetArchTest) that enforce the dependency rules, and write ADR-003 (outbox versus publishing from the API). *Done when:* CI runs them green, and an architecture test fails if `Lib` references `Data`.

Week 4: Hardening the accept path

Goal: reject bad requests early and protect the service from overload. **Learn:** JSON Schema; token-bucket rate limiting; quotas; OpenAPI examples.

- [ ] **Day 1 · Template checks at accept time.** Load the template version, return 404 if it is missing or unpublished, and validate `data` against its JSON Schema using a JSON Schema library whose license you have checked. *Done when:* a missing or mistyped variable returns 422 naming the field.

- [ ] **Day 2 · Rate limiting.** Add the ASP.NET Core rate limiter as a token bucket partitioned by client, with the limit stored on the client row. Return 429 with `Retry-After`. *Done when:* a test that sends faster than the limit sees some 429s with a correct `Retry-After`, and changing the limit in the database takes effect.
- [ ] **Day 3 · Daily quota and expiry.** Add a per-client daily send count in PostgreSQL that returns 429 when exhausted. Accept an optional `expiresAt` and reject one already in the past. *Done when:* tests cover an exhausted quota and the reset at midnight UTC.
- [ ] **Day 4 · Make the contract excellent.** Add request examples, descriptions and every error response (400, 401, 403, 404, 409, 413, 422, 429, 503) to the OpenAPI document. Turn on build-time generation with `OpenApiGenerateDocumentsOnBuild` and commit the generated file. *Done when:* Scalar's request builder can send a valid request, and every documented error can be triggered.
- [ ] **Day 5 · Consolidate: demo and review.** Run the whole accept path by hand through Scalar and a SQL client. Write ADR-004 (keep the `v1.1` route, version by URL segment) and a learning-log note answering: what exactly does a 202 promise? *Done when:* you can narrate a request from HTTP to notification row to outbox row without notes. Milestone 2 is complete.

Weeks 5 to 7: the send pipeline

Week 5: Relay and consumer

Goal: a message travels from the outbox into Kafka and into a consumer. **Learn:** hosted services; idempotent producers and `acks=all`; consumer groups and manual offset commits; `FOR UPDATE SKIP LOCKED`; the claim-check pattern.

- [ ] **Day 1 · Worker skeleton.** Create `NotificationWorker` from the Worker Service template. Add configuration, Serilog, a small health signal, and graceful shutdown handling, and add it to Compose. *Done when:* it starts, logs, and on `docker compose stop` logs a clean shutdown within 30 seconds.
- [ ] **Day 2 · Message contract and topics.** In `Lib/Contracts`, define the Kafka envelope: `schemaVersion`, `notificationId`, `correlationId`, `clientId`, `attempt`, `notBefore`. It carries no recipients and no template data (claim check). Put a topic-provisioning script in `infra/kafka` for `notifications.transactional`, the four retry topics and `notifications.dlq`. *Done when:* a one-shot Compose job creates the topics and `kafka-topics --describe` shows the planned partitions.

- [ ] **Day 3 · The outbox relay.** In `Worker/Outbox`, write a loop that selects unpublished rows with `FOR UPDATE SKIP LOCKED`, publishes with Confluent.Kafka (idempotent producer, `acks=all`), and marks a row published only after the broker acknowledges it. Move the notification from Queued to Published through the state machine. Back off when the outbox is empty. *Done when:* a POST to the Sender API produces a message in `notifications.transactional` within a second, and killing the relay before it marks a row loses nothing (the message is published again).
- [ ] **Day 4 · Consumer with manual commit and a lease.** In `Worker/Consumers`, run a consumer group with auto-commit off. For each message, load the notification, compare-and-set Published to Sending with a lease expiry, skip final states, and commit the offset only after the database write. *Done when:* delivering the same message twice causes one state transition and a log line saying the duplicate was skipped.
- [ ] **Day 5 · Consolidate.** Add an integration test for relay plus consumer (Testcontainers with PostgreSQL and Kafka). Write ADR-005 (at-least-once with idempotent consumers) and a learning-log note on why exactly-once email delivery is impossible. *Done when:* the test is green in CI.

Week 6: Sending and retrying

Goal: real email through the pipeline, with failures handled by design. **Learn:** SMTP reply codes; MailKit; transient versus permanent failures; exponential backoff with jitter; pausing a Kafka partition.

- [ ] **Day 1 · Send through MailKit to Mailpit.** In `Worker/EmailProcessing`, load the template version, render it with `Templating`, build the MIME message (HTML plus text alternative, Message-ID derived from the notification ID), and send with TLS settings and timeouts behind an `IEmailSender` interface. *Done when:* a POST makes an email appear in Mailpit with the right subject, body and Message-ID.
- [ ] **Day 2 · Record the outcome.** On success, move Sending to SmtpAccepted and write a `NotificationAttempt` row (attempt number, timestamps, outcome, duration) in one transaction, then commit the Kafka offset. *Done when:* the attempts table shows one row per send, and a `kill -9` between the SMTP send and the commit causes a redelivery that you observe and write down as the known duplicate window.
- [ ] **Day 3 · Failure classification.** In `Domain/Retry`, write a pure function that maps SMTP replies, timeouts and exceptions to Transient, Permanent or Poison, with a table-driven unit test. Cover 421, 450, 550, a refused connection, a timeout and an unreadable

message. *Done when:* every class is tested, and unknown errors default to transient with a retry cap.

- [ ] **Day 4 · Retry topics with delay tiers.** On a transient failure, publish to the next retry topic (1 minute, 5 minutes, 30 minutes, 2 hours) with `attempt + 1` and a `notBefore` time using full jitter, and set the status to Retrying. A retry consumer pauses its partition until `notBefore`. *Done when:* with Mailpit stopped, a notification climbs the tiers (shorten the delays by configuration for the test) and is sent after Mailpit restarts.
- [ ] **Day 5 · Consolidate.** Move the delays and maximum attempts into configuration, test the whole retry flow with a fake `IEmailSender`, and draw the retry timeline with jitter in your learning log. *Done when:* the retry integration test is green in CI.

Week 7: Dead letters, suppression and resilience

Goal: nothing is stuck or lost, and every failure has a defined end. **Learn:** dead-letter queues; circuit breakers; graceful shutdown; fault injection.

- [ ] **Day 1 · Dead letters.** After the maximum attempts, or on a poison message, publish to `notifications.dlq` and set DeadLettered with a reason. *Done when:* a message with invalid JSON and a notification that exhausts its retries both end in the dead-letter topic, and the partition keeps moving.
- [ ] **Day 2 · Suppression and expiry.** Add the `suppressions` table (keyed hashes, never plain addresses). Before sending, check suppression (final state Suppressed) and `expiresAt` (final state Expired). *Done when:* tests cover both final states and prove neither is retried.
- [ ] **Day 3 · Circuit breaker and graceful shutdown.** Wrap the SMTP send in a Polly breaker (through Microsoft.Extensions.Resilience) so that when it opens, consumers pause instead of burning attempts. Make SIGTERM stop polling, finish in-flight sends and release leases. *Done when:* with SMTP down, attempt counts stop rising while the breaker is open, and `docker compose stop` finishes in-flight work first.
- [ ] **Day 4 · Chaos tests, part 1.** Add Toxiproxy to the integration tests and automate: SMTP down then up, SMTP slow enough to time out, worker killed mid-send, and a poison message. *Done when:* each scenario asserts that nothing is lost and the final state is correct.
- [ ] **Day 5 · Chaos tests, part 2, and review.** Automate Kafka down at accept time (the API still returns 202 and the relay drains after recovery) and PostgreSQL down (the API returns 503 and nothing is acknowledged). Write the chaos report table in `docs/`.

Done when: every scenario is listed with a pass. Milestone 3 is complete.

Weeks 8 to 11: security, admin API and observability

Week 8: Security

Goal: only known clients can send, and secrets are never exposed. **Learn:** OAuth 2.0 client credentials; JWT validation; envelope encryption; STRIDE threat modeling.

- [ ] **Day 1 · Keycloak in Compose.** Add Keycloak with a realm imported from `infra/keycloak/realm.json`. Create a confidential client for a demo app with a `notifications:send` scope, plus three demo users (Viewer, Operator, Admin) for later. Request a token with `curl` and decode it locally. *Done when:* you can get a client-credentials token and name its issuer, audience, expiry and client claim.
- [ ] **Day 2 · JWT validation in the Sender API.** Configure JWT bearer authentication (issuer, audience, signing keys from the provider, small clock skew). Map the token's client to the `Client` row and replace the hard-coded client. Add a scope policy. *Done when:* no token gives 401, a wrong scope gives 403, an inactive client gives 403, and the notification is stored under the right client.
- [ ] **Day 3 · Authorization tests.** Automate a matrix of endpoints against (no token, wrong scope, another client's token). Scope idempotency keys per client, and make `GET /api/v1/notifications/{id}` return 404 for notifications owned by someone else. *Done when:* the matrix runs in CI.
- [ ] **Day 4 · Encrypted secrets and sender profiles.** Write a `SecretProtector` using AES-256-GCM: a data key per secret, wrapped by a master key from the environment, with a key ID stored beside the ciphertext. Add `sender_profiles`, `secrets` and grants tables, and let the worker read a profile's SMTP settings and decrypt the password in memory only. *Done when:* unit tests cover a round trip, tamper detection and rotation by key ID, and the database never holds plaintext.
- [ ] **Day 5 · Consolidate: security hygiene.** Add log redaction with a test that fails if a secret or a full email body is logged, add gitleaks to CI, and add security headers, a request-size limit and HTTPS redirection. Write the STRIDE threat model in `docs/security`. *Done when:* CI is green with all of it. Milestone 4 is complete.

Week 9: Admin API, part 1

Goal: the first half of what the dashboard will call. **Learn:** role-based authorization policies; resource design; optimistic concurrency; template versioning.

- [ ] **Day 1 · Web API skeleton and roles.** Create `NotificationWebApi` with controllers, OpenAPI and Scalar, JWT validation for Keycloak users, and Viewer, Operator and Admin as authorization policies. (The browser cookie flow arrives with the frontend.) *Done when:* a Viewer token gets 403 on an Admin endpoint, and Scalar can send a token.
- [ ] **Day 2 · Clients.** List, create, edit and deactivate clients, including rate limit and daily quota. Credential rotation happens in Keycloak, so document it; never return a secret. *Done when:* integration tests cover each endpoint and role, and the Sender API rejects a deactivated client.
- [ ] **Day 3 · Templates and versions.** Create and edit drafts, publish (immutable afterwards), deprecate, and list versions, with the variable JSON Schema stored per version and optimistic concurrency on edits. *Done when:* a test proves a published version cannot be changed.
- [ ] **Day 4 · Preview and test send.** A preview endpoint renders a version with sample data through `Templating` and returns subject, HTML, text and missing-variable errors. A test-send endpoint sends to an allow-listed address through the normal pipeline. *Done when:* a template with a missing variable returns a clear error, and a test send lands in Mailpit.
- [ ] **Day 5 · Consolidate: paging and audit.** Settle one convention for paging, filtering and error shape, then add the `audit_events` table and write an event for every change from Days 2 to 4. Use an insert-only database role for the audit table. *Done when:* each change creates an audit row, and a test that tries to update or delete one fails.

Week 10: Admin API, part 2

Goal: everything operators need to see and fix. **Learn:** SQL search and indexes; SSRF; safe replays; data retention.

- [ ] **Day 1 · Sender profiles and the SMTP test.** Create, update, rotate and deactivate profiles with a write-only password. The connectivity test resolves the host, refuses loopback, private and link-local addresses unless allow-listed, and sends a controlled test message. *Done when:* tests prove a profile pointing at a private address is refused.
- [ ] **Day 2 · Suppression list.** Search, add, remove (with a reason), and CSV import and export, storing keyed hashes with a pepper from configuration. *Done when:* adding an address and then sending to it ends in Suppressed.
- [ ] **Day 3 · Notification search and detail.** Server-side filter, sort and paging by ID, correlation ID, client, status, date range and recipient (hash lookup, masked display), plus a detail view with the attempt timeline. Add the indexes the queries need and

check the plans with `EXPLAIN`. *Done when:* a search over 100,000 seeded rows uses an index and never a sequential scan.

- [ ] **Day 4 · Retry, replay and unmasking.** Operators can retry an eligible failed notification and replay from the dead-letter topic (a reason is required, and replays are rate-limited). Unmasking a recipient needs the `pii:view` permission and is audited. *Done when:* replaying the same item twice never sends twice, and audit rows exist for all three actions.
- [ ] **Day 5 · Consolidate: retention jobs.** Add scheduled jobs that clear encrypted payloads 7 days after a final state, idempotency keys after 24 hours, and published outbox rows after 24 hours. *Done when:* tests with a fake clock prove each purge.

Week 11: Analytics and observability

Goal: you can see the system, and the dashboard has data to show. **Learn:** OpenTelemetry traces and metrics; percentile SQL; SignalR; consumer lag.

- [ ] **Day 1 · Analytics endpoints.** Counts by status, throughput over time buckets, accept-to-SMTP latency percentiles (using `percentile_cont`), per-client usage and recent errors, all with a time-range parameter. *Done when:* a small hand-calculated dataset produces exactly the expected numbers in a test.
- [ ] **Day 2 · SignalR snapshots.** Add a hub under `Hubs/` that pushes an aggregated snapshot every 2 seconds to authorized users, with the snapshot contract in `Lib`. *Done when:* a test client receives updates, and you can explain why one snapshot per interval protects the server however busy the pipeline is.
- [ ] **Day 3 · Distributed tracing.** Add OpenTelemetry to every service, carry `traceparent` in the Kafka message headers, and export through a collector to Jaeger in an `observability` Compose profile. *Done when:* one trace shows API, outbox relay, consumer and SMTP send.
- [ ] **Day 4 · Metrics.** Expose the `notif_*` metrics from the PRD. A background service measures consumer lag with the Kafka admin client and outbox age, and Prometheus scrapes everything. *Done when:* Prometheus shows lag rising when you stop the worker and falling after you restart it.
- [ ] **Day 5 · Consolidate: alerts, SLOs, runbooks.** Write the alert rules (outbox stuck, lag growing, dead letters present, SMTP degraded, error-budget burn), test them with `promtool test rules`, and write the SLO document and three runbooks. *Done when:* the rule tests pass in CI and each alert links to a runbook. Milestone 5 is complete.

Weeks 12 to 14: load testing and capacity

Week 12: k6 and your first baseline

Goal: real numbers for the accept path, and a worker to automate them. **Learn:** open versus closed workload models; percentiles; thresholds; why a slow server can hide its own overload from a naive test.

- [ ] **Day 1 · k6 smoke test.** Install k6 and write `load-tests/scenarios/smoke.js`: fetch a token in `setup()`, send valid requests with unique idempotency keys, and check for 202. Read the URL and credentials from environment variables, never from files. *Done when:* a run against Compose passes 100% of its checks.
- [ ] **Day 2 · Arrival-rate scenarios and thresholds.** Write `constant-arrival-rate` and `ramping-arrival-rate` scenarios with thresholds (error rate below 1%, p95 below 200 ms, no dropped iterations), using a synthetic client with a high rate limit. *Done when:* you can show a run where the server slows down while the offered rate stays constant, and explain what a closed model (`ramping-vus`) would have done instead.
- [ ] **Day 3 · Your first baseline.** Run a staircase by hand. Record the hardware, versions, container limits and results in `docs/performance.md`, and keep the k6 summary files. *Done when:* the document names the highest passing step and what saturated first (watch `docker stats`).
- [ ] **Day 4 · LoadTestWorker and its data model.** Create `LoadTestWorker` and add the `load_test_configs`, `load_test_runs`, `load_test_buckets` and `load_test_baselines` tables. The worker claims Queued runs with `SKIP LOCKED` and updates a heartbeat. *Done when:* inserting a run row makes the worker claim it, mark it Running and keep heartbeating.
- [ ] **Day 5 · Consolidate: start the metric-transport spike.** Run k6 with `--out json=...` at your baseline rate and measure how much CPU k6 uses. Write the ADR listing the options (tail the JSON output, Prometheus remote write, k6's local REST API) and your first measurement. *Done when:* the ADR is committed.

Week 13: Orchestration

Goal: run k6 safely from the worker and capture its metrics. **Learn:** process management and signals; stream parsing; windowed percentiles; the Prometheus HTTP API.

- [ ] **Day 1 · Templates, validation, caps, allow-list.** The worker builds each run from a versioned scenario template. Parameters (rate, duration, virtual users, payload size) are validated against hard caps per environment, and the target host is checked against an

allow-list when the run starts. No user scripts and no free-form arguments. *Done when:* tests prove an over-cap or non-allow-listed request is rejected before any process starts.

- [ ] **Day 2 · Run k6 as a child process.** Pass parameters and secrets as environment variables, capture the exit code and redacted stderr, and cancel with SIGTERM followed by SIGKILL after a grace period. *Done when:* cancelling stops k6 within the grace period, and killing the worker leaves the run Incomplete after the heartbeat times out (a reaper closes it).
- [ ] **Day 3 · Metrics into aggregates.** Tail k6's JSON output, compute 1-second windows, and store 5-second aggregates (requests per second, p50, p95, p99, errors, virtual users) in `load_test_buckets`, keeping memory use bounded. *Done when:* your aggregates match k6's end-of-test summary within a small tolerance.
- [ ] **Day 4 · Backend metrics sampling.** During a run, query Prometheus for outbox age, consumer lag, processing rate, CPU and memory, and store the samples next to the buckets. *Done when:* a run shows lag rising once the pipeline saturates.
- [ ] **Day 5 · Consolidate.** Add an integration test that runs a 10-second smoke test through the worker against the Compose stack, and update `docs/performance.md`. *Done when:* the test is green.

Week 14: Endpoints, safety and capacity

Goal: finish the load-testing backend and publish your capacity report. **Learn:** staircase search; confirmation holds; regression thresholds.

- [ ] **Day 1 · Verdicts and endpoints.** Compute threshold results and the final status (Passed, Failed, Cancelled or Incomplete). Add Web API endpoints to create, list, cancel and read results for load tests, restricted to a LoadTester role. Starting a test queues a run and returns 202. *Done when:* the endpoints work from Scalar and a Viewer cannot start a test.
- [ ] **Day 2 · Safety rules.** Enforce that synthetic clients can use only the Mailpit sender profile, in code and in a test. Tag test traffic with the run ID and exclude it from analytics by default, audit every start and cancel, and allow one run at a time. *Done when:* a test that points a synthetic client at a real profile fails.
- [ ] **Day 3 · Stress and spike scenarios.** Add the breakpoint staircase and the spike scenario, run each once, and note the bottleneck and the recovery time in `docs/performance.md`. *Done when:* both runs are stored and described.

- [ ] **Day 4 · Soak and capacity search.** Add the soak scenario and automate the PRD's capacity search: staircase, 30-minute confirmation hold, recovery check at 50%, and a recommendation of 70% of the sustainable rate. *Done when:* one automated search finishes and stores the sustainable rate and the bottleneck.
- [ ] **Day 5 · Consolidate: capacity report and regression.** Pin a baseline run, add a comparison that flags p95 more than 15% worse or capacity more than 10% lower, and add a pull-request performance smoke job to CI with relative thresholds. Write the capacity report. *Done when:* a deliberate 50 ms slowdown is flagged as a regression and the report is committed. Milestone 6 is complete.

After the backend: frontend and release

The frontend can start once Week 11 is done, because the admin API and its OpenAPI document exist by then. Doing it alongside Weeks 12 to 14 is fine if you want a change of pace. When you reach it, ask me and I will break each step below into daily tasks like the weeks above.

Frontend track (about 6 to 8 weeks)

1. Setup (3 days): Vite, React and strict TypeScript; Tailwind with shadcn/ui; TanStack Query; routing; an API client generated from the committed OpenAPI documents; linting and Vitest.
2. Sign-in (3 to 4 days): the backend-for-frontend flow in NotificationWebApi (Authorization Code with PKCE, an HttpOnly session cookie, anti-forgery tokens) and role-aware navigation.
3. Notification explorer (1 week): server-side search, filters, paging and the attempt timeline.
4. Clients, templates, sender profiles, suppression and audit log (2 weeks): forms with validation, the template editor with live preview and version history.
5. Overview dashboard (1 week): tiles and charts with Recharts, live updates over SignalR with a polling fallback.
6. Load-test pages (1 to 1.5 weeks): configure, watch live, cancel, compare and read capacity reports.
7. Quality (1 week): accessibility checks with axe, Playwright end-to-end tests and a bundle-size budget.

Release week (5 days)

- [ ] Day 1: a seeder that creates demo clients, templates and 30 days of history, so the dashboard is alive on first launch.
- [ ] Day 2: README with architecture diagram and screenshots; a short demo video.
- [ ] Day 3: finish ADRs, runbooks and the threat model; run the security scans and fix findings.
- [ ] Day 4: publish the capacity report and chaos report; add CI badges.
- [ ] Day 5: tag `v1.0.0` and publish the container images.

Stretch goals once it is all working: monthly partitioning of the notifications table, Kubernetes with KEDA scaling consumers on lag, and OpenBao for secrets.

Sources, checked 9 October 2026

- .NET and .NET Core support policy: .NET 10 LTS support until 14 November 2028; .NET 8 and 9 until 10 November 2026.
- OpenAPI support in ASP.NET Core API apps: `Microsoft.AspNetCore.OpenApi`, `AddOpenApi`, `MapOpenApi`, build-time generation.
- Scalar for .NET: `MapScalarApiReference`, the `/scalar` route, MIT license.
- Notification Platform PRD v2.0: the requirements this plan builds.